#ERRORS: 1.wrong video file path 2.large high-resolution videos caused memory-related issues, which can be handled by resizing frames before processing it

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import
preprocess_input, decode_predictions

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

import cv2
import numpy as np

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Handle videos where frame count is 0
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  #
Important fix
            frames.append(frame)

        count += 1

        if len(frames) >= max_frames:
            break
```

```
    cap.release()
    print("Extracted frames:", len(frames))
    return np.array(frames)



def classify_video(video_path):
    # The 'frames' variable here is local to this function
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames)

    # Average predictions across frames
    avg_pred = np.mean(predictions, axis=0)
    #top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0),
top=1)

    top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0),
top=5)

    for pred in top_pred[0]:
      print(pred[1], ":", round(pred[2]*100, 2), "%")


    print("Final Prediction:", top_pred[0][0][1])

# Replace with your video path
video_path = "/content/10994870-hd_1080_1920_25fps.mp4"
classify_video(video_path)

# Make 'frames' globally accessible for further inspection
frames = extract_frames(video_path) # This line is added
```

```
Extracted frames: 20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
velvet : 24.06 %
window_screen : 20.68 %
wool : 6.45 %
shower_curtain : 2.59 %
quilt : 1.58 %
Final Prediction: velvet
Extracted frames: 20
```

```
import cv2

# Open video file
```

```python
classifier = cv2.VideoCapture(video_path)

frame_count = 0

while True:
    ret, frame = classifier.read()

    if not ret:
        break

    frame_count += 1
    print(f"Frame {frame_count} Shape: {frame.shape}")

classifier.release()
print("Finished reading video.")
```

```
Frame 1 Shape: (1920, 1080, 3)
Frame 2 Shape: (1920, 1080, 3)
Frame 3 Shape: (1920, 1080, 3)
Frame 4 Shape: (1920, 1080, 3)
Frame 5 Shape: (1920, 1080, 3)
Frame 6 Shape: (1920, 1080, 3)
Frame 7 Shape: (1920, 1080, 3)
Frame 8 Shape: (1920, 1080, 3)
Frame 9 Shape: (1920, 1080, 3)
Frame 10 Shape: (1920, 1080, 3)
Frame 11 Shape: (1920, 1080, 3)
Frame 12 Shape: (1920, 1080, 3)
Frame 13 Shape: (1920, 1080, 3)
Frame 14 Shape: (1920, 1080, 3)
Frame 15 Shape: (1920, 1080, 3)
Frame 16 Shape: (1920, 1080, 3)
Frame 17 Shape: (1920, 1080, 3)
Frame 18 Shape: (1920, 1080, 3)
Frame 19 Shape: (1920, 1080, 3)
Frame 20 Shape: (1920, 1080, 3)
Frame 21 Shape: (1920, 1080, 3)
Frame 22 Shape: (1920, 1080, 3)
Frame 23 Shape: (1920, 1080, 3)
Frame 24 Shape: (1920, 1080, 3)
Frame 25 Shape: (1920, 1080, 3)
Frame 26 Shape: (1920, 1080, 3)
Frame 27 Shape: (1920, 1080, 3)
Frame 28 Shape: (1920, 1080, 3)
Frame 29 Shape: (1920, 1080, 3)
Frame 30 Shape: (1920, 1080, 3)
Frame 31 Shape: (1920, 1080, 3)
Frame 32 Shape: (1920, 1080, 3)
Frame 33 Shape: (1920, 1080, 3)
Frame 34 Shape: (1920, 1080, 3)
```

```
Frame 35 Shape: (1920, 1080, 3)
Frame 36 Shape: (1920, 1080, 3)
Frame 37 Shape: (1920, 1080, 3)
Frame 38 Shape: (1920, 1080, 3)
Frame 39 Shape: (1920, 1080, 3)
Frame 40 Shape: (1920, 1080, 3)
Frame 41 Shape: (1920, 1080, 3)
Frame 42 Shape: (1920, 1080, 3)
Frame 43 Shape: (1920, 1080, 3)
Frame 44 Shape: (1920, 1080, 3)
Frame 45 Shape: (1920, 1080, 3)
Frame 46 Shape: (1920, 1080, 3)
Frame 47 Shape: (1920, 1080, 3)
Frame 48 Shape: (1920, 1080, 3)
Frame 49 Shape: (1920, 1080, 3)
Frame 50 Shape: (1920, 1080, 3)
Frame 51 Shape: (1920, 1080, 3)
Frame 52 Shape: (1920, 1080, 3)
Frame 53 Shape: (1920, 1080, 3)
Frame 54 Shape: (1920, 1080, 3)
Frame 55 Shape: (1920, 1080, 3)
Frame 56 Shape: (1920, 1080, 3)
Frame 57 Shape: (1920, 1080, 3)
Frame 58 Shape: (1920, 1080, 3)
Frame 59 Shape: (1920, 1080, 3)
Frame 60 Shape: (1920, 1080, 3)
Frame 61 Shape: (1920, 1080, 3)
Frame 62 Shape: (1920, 1080, 3)
Frame 63 Shape: (1920, 1080, 3)
Frame 64 Shape: (1920, 1080, 3)
Frame 65 Shape: (1920, 1080, 3)
Frame 66 Shape: (1920, 1080, 3)
Frame 67 Shape: (1920, 1080, 3)
Frame 68 Shape: (1920, 1080, 3)
Frame 69 Shape: (1920, 1080, 3)
Frame 70 Shape: (1920, 1080, 3)
Frame 71 Shape: (1920, 1080, 3)
Frame 72 Shape: (1920, 1080, 3)
Frame 73 Shape: (1920, 1080, 3)
Frame 74 Shape: (1920, 1080, 3)
Frame 75 Shape: (1920, 1080, 3)
Frame 76 Shape: (1920, 1080, 3)
Frame 77 Shape: (1920, 1080, 3)
Frame 78 Shape: (1920, 1080, 3)
Frame 79 Shape: (1920, 1080, 3)
Frame 80 Shape: (1920, 1080, 3)
Frame 81 Shape: (1920, 1080, 3)
Frame 82 Shape: (1920, 1080, 3)
Frame 83 Shape: (1920, 1080, 3)
```

```
Frame 84 Shape: (1920, 1080, 3)
Frame 85 Shape: (1920, 1080, 3)
Frame 86 Shape: (1920, 1080, 3)
Frame 87 Shape: (1920, 1080, 3)
Frame 88 Shape: (1920, 1080, 3)
Frame 89 Shape: (1920, 1080, 3)
Frame 90 Shape: (1920, 1080, 3)
Frame 91 Shape: (1920, 1080, 3)
Frame 92 Shape: (1920, 1080, 3)
Frame 93 Shape: (1920, 1080, 3)
Frame 94 Shape: (1920, 1080, 3)
Frame 95 Shape: (1920, 1080, 3)
Frame 96 Shape: (1920, 1080, 3)
Frame 97 Shape: (1920, 1080, 3)
Frame 98 Shape: (1920, 1080, 3)
Frame 99 Shape: (1920, 1080, 3)
Frame 100 Shape: (1920, 1080, 3)
Frame 101 Shape: (1920, 1080, 3)
Frame 102 Shape: (1920, 1080, 3)
Frame 103 Shape: (1920, 1080, 3)
Frame 104 Shape: (1920, 1080, 3)
Frame 105 Shape: (1920, 1080, 3)
Frame 106 Shape: (1920, 1080, 3)
Frame 107 Shape: (1920, 1080, 3)
Frame 108 Shape: (1920, 1080, 3)
Frame 109 Shape: (1920, 1080, 3)
Frame 110 Shape: (1920, 1080, 3)
Frame 111 Shape: (1920, 1080, 3)
Frame 112 Shape: (1920, 1080, 3)
Frame 113 Shape: (1920, 1080, 3)
Frame 114 Shape: (1920, 1080, 3)
Frame 115 Shape: (1920, 1080, 3)
Frame 116 Shape: (1920, 1080, 3)
Frame 117 Shape: (1920, 1080, 3)
Frame 118 Shape: (1920, 1080, 3)
Frame 119 Shape: (1920, 1080, 3)
Frame 120 Shape: (1920, 1080, 3)
Frame 121 Shape: (1920, 1080, 3)
Frame 122 Shape: (1920, 1080, 3)
Frame 123 Shape: (1920, 1080, 3)
Frame 124 Shape: (1920, 1080, 3)
Frame 125 Shape: (1920, 1080, 3)
Frame 126 Shape: (1920, 1080, 3)
Finished reading video.
```

```
import cv2

classifier = cv2.VideoCapture(video_path)

ret, frame = classifier.read()
```

```python
if ret:
    print("Frame Shape:", frame.shape)
else:
    print("Could not read frame")

classifier.release()
```

```
Frame Shape: (1920, 1080, 3)
```

```python
# Display the raw pixel matrix (the 'frame' variable)
display(frame)
```

```
array([[[  7,  13,  35],
        [ 55,  61,  83],
        [ 19,  24,  53],
        ...,
        [ 38,  38,  36],
        [ 26,  26,  24],
        [ 13,  13,  11]],

       [[ 44,  50,  72],
        [ 87,  93, 115],
        [ 47,  52,  81],
        ...,
        [ 76,  76,  74],
        [ 64,  64,  62],
        [ 48,  48,  46]],

       [[ 11,  23,  20],
        [ 51,  63,  60],
        [ 12,  22,  26],
        ...,
        [ 65,  65,  63],
        [ 46,  46,  44],
        [ 29,  29,  27]],

       ...,

       [[100,  98, 105],
        [243, 241, 248],
        [255, 255, 255],
        ...,
        [ 37,  32,  31],
        [ 24,  19,  18],
        [ 17,  12,  11]],

       [[100,  98, 105],
        [228, 226, 233],
        [252, 250, 255],
        ...,
```

```
        [ 30,  25,  24],
        [ 20,  15,  14],
        [ 15,  10,   9]],

       [[100,  98, 105],
        [206, 204, 211],
        [196, 194, 201],
        ...,
        [ 23,  18,  17],
        [ 14,   9,   8],
        [  9,   4,   3]]], dtype=uint8)
```

```
# Example of pixel code:

# Accessing the pixel value at row 0, column 0 (top-left corner)
top_left_pixel = frame[0, 0]
print(f"Top-left pixel (R, G, B): {top_left_pixel}")

# Accessing the pixel value at the center of the frame
center_row, center_col = frame.shape[0] // 2, frame.shape[1] // 2
center_pixel = frame[center_row, center_col]
print(f"Center pixel (R, G, B): {center_pixel}")

# Changing the color of a specific pixel (e.g., make top-left pixel
red)
# Note: This will modify the 'frame' array in memory. If you want to
visualize
# this change, you would need to save or display the modified frame.
frame_copy = frame.copy() # Make a copy to avoid modifying the
original frame
frame_copy[0, 0] = [255, 0, 0] # Set R=255, G=0, B=0 for the top-left
pixel
print(f"Modified top-left pixel (R, G, B): {frame_copy[0, 0]}")

# You can also perform operations on entire color channels or regions.
# For example, converting to grayscale (simple average method):
grayscale_frame = np.mean(frame, axis=2).astype(np.uint8)
print(f"Shape of grayscale frame: {grayscale_frame.shape}")

# Displaying the first few rows and columns of the grayscale frame's
pixel values
print("First 5x5 pixels of grayscale frame:")
print(grayscale_frame[:5, :5])
```

```
Top-left pixel (R, G, B): [ 7 13 35]
Center pixel (R, G, B): [15 21 16]
Modified top-left pixel (R, G, B): [255   0   0]
Shape of grayscale frame: (1920, 1080)
First 5x5 pixels of grayscale frame:
[[18 66 32 12 13]
```

```
 [55 98 60 45 55]
 [18 58 20 15 31]
 [ 0 35  2  2 10]
 [ 0 49  9  3 10]]
```

```
print(f"Shape of 'frame': {frame.shape}")
print(f"First element (row) of 'frame':\n{frame[0]}\n")

# Check if 'frames' variable exists from previous execution of
classify_video
if 'frames' in globals() and isinstance(frames, np.ndarray):
    print(f"Shape of 'frames' (array of extracted frames):
{frames.shape}")
    # Displaying the first pixel of the first frame for brevity
    print(f"First element (first frame's top-left pixel) of 'frames':\
n{frames[0][0][0]}\n")
else:
    print("'frames' variable not found or is not a NumPy array. Please
ensure the 'classify_video' function or 'extract_frames' function has
been executed successfully.")
```

```
Shape of 'frame': (1920, 1080, 3)
First element (row) of 'frame':
[[ 7 13 35]
 [55 61 83]
 [19 24 53]
 ...
 [38 38 36]
 [26 26 24]
 [13 13 11]]

Shape of 'frames' (array of extracted frames): (20, 224, 224, 3)
First element (first frame's top-left pixel) of 'frames':
[16 13  4]
```

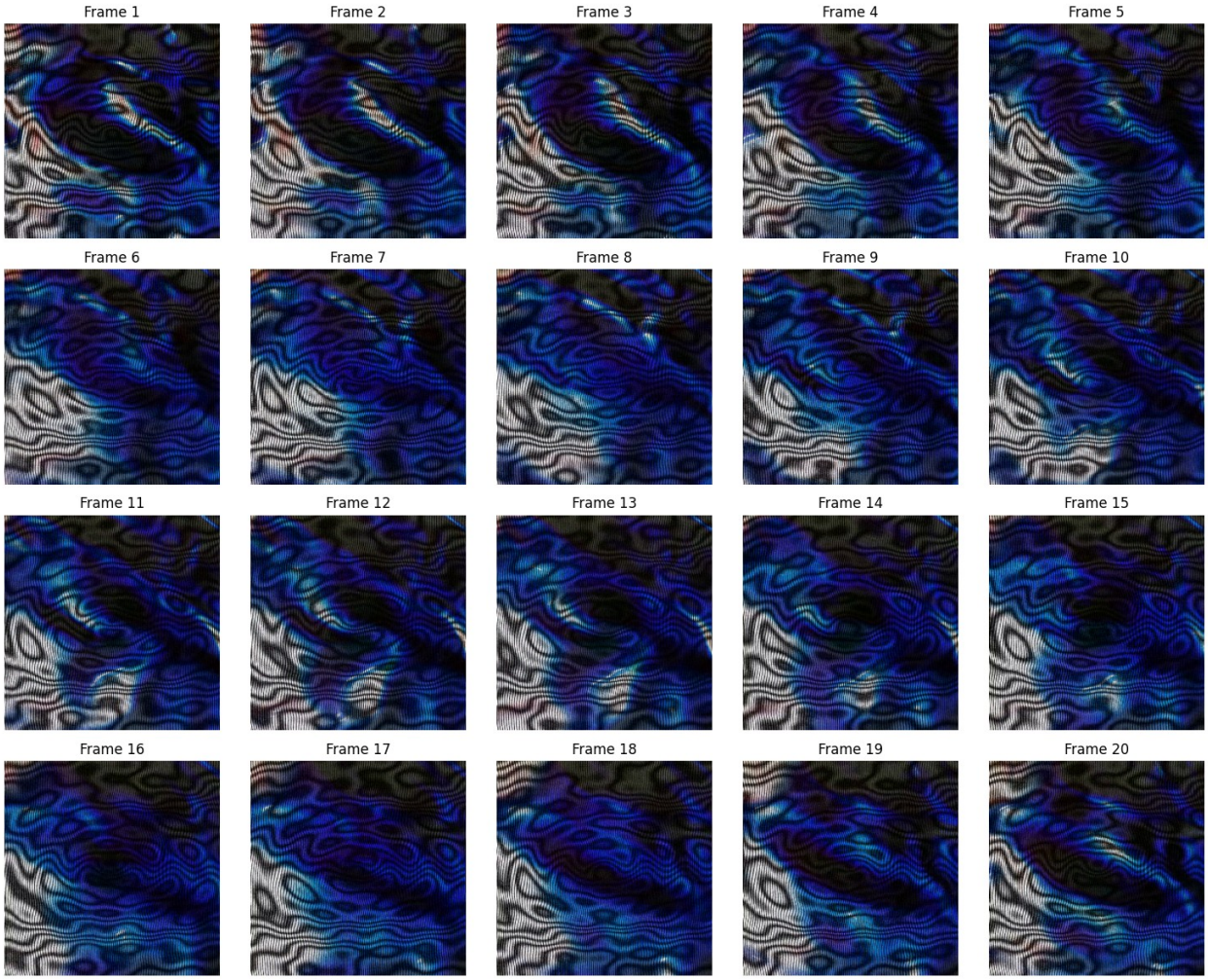
```
frame[0]
```

```
array([[ 7, 13, 35],
       [55, 61, 83],
       [19, 24, 53],
       ...,
       [38, 38, 36],
       [26, 26, 24],
       [13, 13, 11]], dtype=uint8)
```

```
import matplotlib.pyplot as plt
import numpy as np
```

```python
if len(frames) > 0:
    num_frames_to_display = min(20, len(frames))
    print(f"Displaying the first {num_frames_to_display} extracted
frames and their pixel value summaries:\n")

    rows = (num_frames_to_display + 4) // 5
    cols = 5

    plt.figure(figsize=(cols * 3, rows * 3))
    for i in range(num_frames_to_display):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(frames[i])
        plt.title(f"Frame {i+1}")
        plt.axis('off')

        min_val = np.min(frames[i])
        max_val = np.max(frames[i])
        mean_val = np.mean(frames[i])
        print(f"Frame {i+1} - Min Pixel: {min_val}, Max Pixel:
{max_val}, Mean Pixel: {mean_val:.2f}")

    plt.tight_layout()
    plt.show()

else:
    print("No frames available to display images or pixel values.")
```

```
Displaying the first 20 extracted frames and their pixel value
summaries:

Frame 1 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.54
Frame 2 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 51.44
Frame 3 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.02
Frame 4 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 50.63
Frame 5 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 50.49
Frame 6 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 52.69
Frame 7 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 52.40
Frame 8 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 51.64
Frame 9 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.14
Frame 10 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.38
Frame 11 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 50.57
Frame 12 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 48.73
Frame 13 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.85
Frame 14 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 48.56
Frame 15 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 49.61
Frame 16 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 48.40
Frame 17 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 50.38
Frame 18 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 51.63
Frame 19 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 52.66
Frame 20 - Min Pixel: 0, Max Pixel: 255, Mean Pixel: 50.32
```


Frame 1
Frame 2
Frame 3
Frame 4
Frame 5
Frame 6
Frame 7
Frame 8
Frame 9
Frame 10
Frame 11
Frame 12
Frame 13
Frame 14
Frame 15
Frame 16
Frame 17
Frame 18
Frame 19
Frame 20